Modern Java Cheat Sheet

Resources

- Java 8 Javadoc
- Java SE8 for the Really Impatient: A Short Course on the Basics by Cay S. Horstmann
- Java 8 in Action: Lambdas, streams, and functional-style programming by Raoul-Gabriel Urma, Mario Fusco, and Alan Mycroft
- http://horstmann.com/heig-vd/spring2015/poo/

Java 8 Optional

Optional is a functional replacement for null. Javadoc:
https://docs.oracle.com/javase/8/docs/api/java/util/Optional.html

Java Optional Method Summary

Method	Description
empty()	create Optional with an empty value
of(T)	create an optional with a non-null value
ofNullable(T)	create an optional with a possibly null reference
ifPresent(Consumer<? super T>)	if the underlying value is not empty, execute the Consumer with it as an argument
isPresent()	if value is not emtpy, return true
get()	get the underlying value, or if null, throw NoSuchElementException
orElseGet(T)	get the underlying value, or if null, return the argument. Argument can be null.
orElse(Supplier<? extends T>)	get the underlying value, or if null, execute the Supplier function and return the result
orElseThrow(Supplier<? extends X>)	get the underlying value, or if null, execute the Supplier function and throw the result
filter(Predicate<? super T>)	if the predicate is true, return the Optional, other wise return empty Optional

Method	Description
map(Function<? super T,? extends U>)	apply the Function, and return the non-null result auto-wrapped as an Optional, or if null an empty Optional
flatMap(Function<? super T, Optional<U>>)	apply the Optional-bearing Function, and return without wrapping as an Optional

Tips

- Prefer ifPresent over isPresent/get

```
// length of the value or "" if nothing
int length = res.orElse("").length();

// run the lambda if there is a value
res.ifPresent(v -> results.add(v));
```

Return an Optional

```
Optional<Double> squareRoot(double x) {
   if (x >= 0) { return Optional.of(Math.sqrt(x)); }
   else { return Optional.empty(); }
}
```

Uses of filter, map, and flatMap

```
System.out.println("empty().filter(\"foo\"::equals)" + " => " + empty().filter("foo"::eq
// empty().filter("foo"::equals) => Optional.empty

System.out.println("of(\"foo\").filter(\"foo\"::equals)" + " => " + of("foo").filter("fo
// of("foo").filter("foo"::equals) => Optional[foo]

System.out.println("ofNullable(null).filter(\"foo\"::equals)" + " => " + ofNullable(null
// ofNullable(null).filter("foo"::equals) => Optional.empty

System.out.println("empty().map(\"foo\"::equals)" + " => " + empty().map("foo"::equals))
// empty().map("foo"::equals) => Optional.empty

System.out.println("of(\"foo\").map(\"foo\"::equals)" + " => " + of("foo").map("foo"::eq
// of("foo").map("foo"::equals) => Optional[true]

System.out.println("ofNullable(null).map(\"foo\"::equals)" + " => " + ofNullable(null).m
// ofNullable(null).map("foo"::equals) => Optional.empty

System.out.println("empty().flatMap(s -> ofNullable(\"foo\".equals(s)))" + " => " + empt
// empty().flatMap(s -> ofNullable("foo".equals(s))) => Optional.empty

System.out.println("of(\"foo\").flatMap(s -> ofNullable(\"foo\".equals(s)))" + " => " +
// of("foo").flatMap(s -> ofNullable("foo".equals(s))) => Optional[true]

System.out.println("ofNullable(null).flatMap(s -> ofNullable(\"foo\".equals(s)))" + " =>
// ofNullable(null).flatMap(s -> ofNullable("foo".equals(s))) => Optional.empty
```

Lambda Expressions

Used to contruct implementations of Functional Interfaces without explicitly creating new classes and instances. All functional interfaces implement exactly one method, which allows for syntactic sugar to implement that one method with much less code.

```
() -> expression | statement
param -> expression | statement
(params) -> expression | statement
(params) -> { expressions and statements }
(types) (params) -> { expressions and statements }
```

- optional types before parameters
- final and annotations can be used on typed parameters

```
() -> System.out.println("I'm a function that prints this statement.");
```

```
a -> a * 2        // calculate double of a (idiomatic)
(a) -> a * 2      // without the type, with parens
(int a) -> a * 2  // with type and parens
(final int a) -> a * 2  // with final, type and parens (final only works with a type)
(@NonNull int a) -> a * 2  // with an annotation, type and parens (annotations only work
```

```
(a, b) -> a + b   // Sum of 2 parameters, inferred types (idiomatic)
(int a, int b) -> a + b   // Sum of 2 parameters, explicit types
(int, int) (tx, status) -> a + b  // explicit types, alternate syntax
```

If the lambda is more than one expression, we must use `{ }` and `return`

```
(x, y) -> {
        int sum = x + y;
        int avg = sum / 2;
        return avg;
}
```

Within a lambda, you can reference only parameters and final variables from the outer scope.

A lambda expression cannot stand alone in Java, it need to be associated to a functional interface.

```
@FunctionalInterface
interface MyMath {
    int getDoubleOf(int a);
}

MyMath d = a -> a * 2; // associated to the interface
d.getDoubleOf(4); // is 8
```

Useful functions

Return the input parameter

```
x -> x
// or
Function.identity()
```

Method and Constructor References

Allows referencing methods and constructors without writing an explicit lambda function

```
// Lambda Form:
getPrimes(numbers, a -> StaticMethod.isPrime(a));

// Method Reference:
getPrimes(numbers, StaticMethod::isPrime);
```

Style	Method Reference	Lambda Form
Class::staticMethod	`MyClass::staticMethod`	`n -> MyClass.staticMethod(n)`
Class::instanceMethod	`String::toUpperCase`	`(String w) -> w.toUpperCase()`
...	`String::compareTo`	`(String s, String t) -> s.compareTo(t)`
...	`System.out::println`	`x -> System.out.println(x)`
instance::instanceMethod	`manager::getByID`	`(int id) -> manager.getByID(id)`
this::instanceMethod	`this::getValueByID`	`(int id) -> this.getValueByID(id)`
super::instanceMethod	`super::aMethodThatIveOverridden`	`(int id) -> super.aMethodThatIveOverridden(id)`
Class::new	`Double::new`	`n -> new Double(n)`
Class[]::new	`String[]::new`	`(int n) -> new String[n]`
primitive[]::new	`int[]::new`	`(int n) -> new int[n]`

Examples of static methods commonly used:

- System.out::println
- Math::pow
- StringUtils::isBlank (though s -> isBlank(s) with a static import is often more concise)

Generified Functional Interfaces

Class	Description	Single method	Additional methods
Function<T,R>	a function that accepts one argument and produces a result	R apply(T t)	andThen(Function), compose(Function), static identity()
Predicate	a boolean-valued function of one argument	boolean test(T t)	static isEqual(Object), and(Predicate), or(Predicate), negate()
Supplier	a supplier of values.	T get()	n/a
Consumer	an operation that accepts a single input argument and returns no result	void accept(T)	andThen(Consumer)
Runnable	an executable unit that takes no arguments and returns no	void run()	n/a

Class	Description	Single method	Additional methods
	result		
Callable	an executable unit that returns a result. throws Exception. Typically used when the execution side-effects are purpose rather than the return value.	V call()	n/a
UnaryOperator	an operation on a single operand that produces a result of the same type as its operand.	R apply(T t)	static identity()
BiConsumer<T,U>	an operation that accepts two input arguments and returns no result	void accept(T t, U u)	andThen(BiConsumer)
BiFunction<T,U,R>	a function that accepts two arguments and produces a result.	R apply(T t, U u)	andThen(Function)
BinaryOperator	an operation upon two operands of the same type, producing a result of the same type as the operands.	R apply(T t, U u)	andThen(Function)
BiPredicate<T,U>	a predicate (boolean-valued function) of two arguments.	boolean test(T t, U u)	and(BiPredicate), or(BiPredicate), negate()

Most interfaces also have a primitive-specific variant, to avoid autoboxing that that would happen were the generic functional interface version used.

Recommended:

- Supplier<? extends T>
- Consumer<? super T>

Class	Description
{Int,Long,Double}Function	a function that accepts an type-valued argument and produces a result.
{Int,Long,Double}Predicate	a predicate (boolean-valued function) of one long-valued argument.
{Int,Long,Double}Consumer	an operation that accepts a single type-valued argument and returns no result.
{Boolean,Int,Long,Double}Supplier	a supplier of type-valued results.
To{Int,Long,Double}Function	a function that produces a type-valued result.

Class	Description
To{Int,Long,Double}BiFunction<T,U>	a function that accepts two arguments and produces an type-valued result.
{Int,Long,Double}UnaryOperator	an operation on a single type-valued operand that produces a type-valued result.
{Int,Long,Double}BinaryOperator	an operation upon two type-valued operands and producing a type-valued result.
{Int,Long,Double}To{Int,Long,Double}Function	a function that accepts a type1-valued argument and produces an type2-valued result. If type1 == type2, use UnaryOperator variant instead.
Obj{Int,Long,Double}Consumer	an operation that accepts an object-valued and a type-valued argument, and returns no result.

Streams

https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html

Similar to collections, but

- Data representation rather than data store
- *immutable* (produce new streams)
- *lazy* (only computes what is necessary)

Process

- **Create** a stream (all Collections now have stream() method, or create using StreamSupport and a spliterator
- **Transform** the stream entries – filter, map, etc
- **Reduce** into a collection of entries or value – collect and Collectors, count

```
Set<String> genres =
        Stream.of("Jazz", "Blues", "Rock")
                .map(String::toLowerCase)
                .collect(Collectors.toSet());
```

Create

```
Stream<String> stream = Stream.of("Jazz", "Blues", "Rock"); // stream from references

Stream<String> stream = Stream.of(myArray); // from an array

list.stream(); // from a list

StreamSupport.stream(iterable.spliterator(), false); // from an iterable

StreamSupport.stream(Spliterators.spliteratorUnknownSize(iterator, Spliterator.ORDERED),

Stream<Integer> integers = Stream.iterate(0, n -> n + 1); // Infinite stream, lazily gen

Stream<String> strings = Arrays.stream(new String[] { "a", "b", "c"});
Stream<String> strings = Stream.of(new String[] { "a", "b", "c"});

IntStream = Arrays.stream(new int[] {1, 2});
Stream<int[]> arr1 = Stream.of(new int[] {1, 2});  // stream of int arrays, not stream o
```

```
// static <T> Stream<T> stream(T[] array, int startInclusive, int endExclusive), also ov
Stream<String> strings = Arrays.stream(new String[] { "a", "b", "c", "d", "e"}, 2, 4) //
```

parallel Returns an equivalent stream that is parallel

Transform

filter(Predicate<? super T>) and *map(Function<? super T,? extends R>)* are the most common. Filter retains elements that match the predicate, typically written as a lambda. Map applies the Function to each element, also typically written as a lambda.

```
// count of names longer than 24 characters
long longNamesCount = list
   .filter(n -> n.length() > 24)
   .map(n -> n * 2)  // doesn't do anything, count is still the same
   .count();

// names longer than 24 characters, with name strings reversed
Set<String> longNamesReversed = list
   .filter(n -> n.length() > 24)
   .map(n -> StringUtils.reverse(n))
   .collect(toSet());
```

filter Stream filter(Predicate<? super T>)

Remove elements not matching the Predicate.

map signature: Stream map(Function<? super T, ? extends R>)

Apply a Function to each element

Apply a Function to each element

// Apply “toLowerCase” for each element List list = Arrays.asList(new String[]{"A", "B", "C"}); res = list.stream().map(w -> w.toLowerCase()); list.stream().map(String::toLowerCase); // stream of ["a", "b", "c"]

Stream.of(1,2,3,4,5).map(x -> x + 1); // stream of [2, 3, 4, 5, 6]

limit `limit(maxSize)` The first *n* elements

```
Stream.of(1,2,3,4,5).limit(3);
// stream of [1, 2, 3]
```

skip Discarding the first *n* elements

```
Stream.of(1,2,3,4,5).skip(3);
// stream of [4, 5]
```

distinct Remove duplicated elements

```
Stream.of(1,0,0,1,0,1).distinct();
// stream of [1, 0]
```

sorted Sort elements (must be *Comparable*)

```
Stream.of(2,1,5,3,4).sorted(); // must be Comparable
//  stream of [1, 2, 3, 4, 5]

// using sorted(Comparator<? super T> comparator)
```

```
Stream.of(2,1,5,3,4).sorted((i1, i2) -> -1 * Integer.compare(i1, i2)); // reverse sort
// stream of [5, 4, 3, 2, 1]
```

Collect

Collect to a collection, a value, or a boolean. Most commonly done with a statically imported method from Collectors

toList and **toSet**

```
// Collect into a List
List<String> myList = stream.collect(toList());

// Collect into a Set
Set<String> mySet = stream.collect(toSet());
```

toArray

```
// Collect into an array
String[] myArray = stream.toArray(String[]::new);
```

toMap

Collectors.toMap(Function keyFunction, Function valueFunction)

```
Map<String, Foo> result =
    choices.stream().collect(Collectors.toMap(Foo::getName,
                                              Function.identity()));
```

Three toMap static methods

- simple key and value – static <T,K,U> Collector<T,?,Map<K,U» toMap(Function<? super T,? extends K> keyMapper, Function<? super T,? extends U> valueMapper)
- key, value, and merge function if there are duplicate keys in the stream – static <T,K,U> Collector<T,?,Map<K,U» toMap(Function<? super T,? extends K> keyMapper, Function<? super T,? extends U> valueMapper, BinaryOperator mergeFunction)
- key, value, merge, and a Supplier to supply the intial map – static <T,K,U,M extends Map<K,U» Collector<T,?,M> toMap(Function<? super T,? extends K> keyMapper, Function<? super T,? extends U> valueMapper, BinaryOperator mergeFunction, Supplier mapSupplier)

Also, all methods have a toMapConcurrent that returns a concurrent map for use with parallel streams, and the Supplier

toCollection

Transform the Stream into a specified Collection

```
// Collect into a specified Collection
ConcurrentSkipListSet<String> mySet = stream.collect(toCollection(ConcurrentSkipListSet:

// Collect into an existing collection
List<?> groups = ...;
stream.collect(toCollection(() -> groups));
```

joining

Concatenate the Stream values into a String.

```
// Collect into a String, concatentating with no delmiter
String str = Stream.of("a", "b", "c").collect(joining());
```

```
//> abc

// Collect into a String, concatenating with a delimter
String str = Stream.of("a", "b", "c").collect(joining(", "));
//> a, b, c

// Collect into a String, concatenating with a delimter, prefix, and suffix
String str = Stream.of("a", "b", "c").collect(joining(", "), "before ", " after");
//> before a, b, c after
```

allMatch All entries of the stream match the predicate. boolean allMatch(Predicate<? super T> predicate)

```
// Check if there is a "e" in each elements
boolean res = words.allMatch(n -> n.contains("e"));
```

anyMatch Any entry in the stream matches the predicate. boolean anyMatch(Predicate<? super T> predicate)

noneMatch No entry in the stream matches the predicate. boolean noneMatch(Predicate<? super T> predicate)

findAny Find any entry in the stream that matches the predicate and return it. Faster than **findFirst** in parallel streams if any entry is sufficient.

```
// Optional<String> contains a string or nothing
Optional<String> res = stream
   .filter(w -> w.length() > 10)
   .findAny();
```

findFirst

```
// Optional<String> contains a string or nothing
Optional<String> res = stream
   .filter(w -> w.length() > 10)
   .findFirst();
```

Finds the first entry the stream that matches, not just any match. The semantics for this still hold even if the stream is parallel, which requires additional operations.

reduce
Reduce the elements to a single value (rarely used, given the other more specific methods) The BiFunction accumulator must be an associative function, e.g., (a op b) op c == a op (b op c)

```
// reduce to an object with the same type as the stream, with the initial value of the a
Optional<String> r1 = Stream.of("a", "b", "c").reduce((acc, e) -> acc + "|" + e.toUpperC
//> A|B|C

// reduce with an explicit initial accumulator state
String r2 = Stream.of("a", "b", "c").reduce("", (acc, e) -> acc + "|" + e.toUpperCase())
//> A|B|C

// fused map / reduce (tbd)
 <U> U reduce(U identity,
              BiFunction<U,? super T,U> accumulator,
              BinaryOperator<U> combiner)
```

Grouping Results

Collectors.groupingBy

collect a stream into a Map grouped by a function.

```
        // I18nEntry has three attributes: locale, key, and value
        List<I18nEntry> list = new ArrayList<>();
        list.add(new I18nEntry(ENGLISH, "key1", "value1"));
        list.add(new I18nEntry(ENGLISH, "key2", "value2"));
        list.add(new I18nEntry(FRENCH, "key3", "value3"));
        list.add(new I18nEntry(FRENCH, "key4", "value4"));
        list.add(new I18nEntry(GERMAN, "key5", "value5"));
        list.add(new I18nEntry(GERMAN, "key6", "value6"));

        Map<String, List<I18nEntry>> mapOfLocaleStringToI18nEntry = list.stream()
                .collect(groupingBy(w -> w.getLocale().toString()));
        System.out.println(mapOfLocaleStringToI18nEntry);
        // {de=[I18nEntry{ de, 'key5', 'value5'}, I18nEntry{ de, 'key6', 'value6'}],
        // en=[I18nEntry{ en, 'key1', 'value1'}, I18nEntry{ en, 'key2', 'value2'}],
        // fr=[I18nEntry{ fr, 'key3', 'value3'}, I18nEntry{ fr, 'key4', 'value4'}]}

        Map<String, Set<String>> mapOfLocaleStringToSetOfI18nEntryNames
                = list.stream().collect(groupingBy(w -> w.getLocale().toString(),
                mapping(I18nEntry::getKey, toSet())));
        System.out.println(mapOfLocaleStringToSetOfI18nEntryNames);
        // {de=[key5, key6], en=[key1, key2], fr=[key3, key4]}

        Map<String, Map<String, String>> mapOfLocaleStringToMapOfKeyValues =
                list.stream().collect(groupingBy(w -> w.getLocale().toString(),
                        toMap(I18nEntry::getKey, I18nEntry::getValue)));
        System.out.println(mapOfLocaleStringToMapOfKeyValues);
        // {de={key5=value5, key6=value6}, en={key1=value1, key2=value2}, fr={key3=value.
```

groupingByConcurrent does the same thing, only concurrently.

Collectors.counting Count the number of values in a group

Collectors.summing{Int|Long|Double} `summingInt`, `summingLong`, `summingDouble` to sum group values

**Collectors.summarizing{Int	Long	Double}**

{Int	Long	Double}SummaryStatistics

Collectors.averaging{Int|Long|Double} `averagingInt`, `averagingLong`, ...

```
// Average length of each element of a group
Collectors.averagingInt(String::length)
```

PS: Don't forget Optional (like `Map<T, Optional<T>>`) with some Collection methods (like `Collectors.maxBy`).

reducing

minBy

maxBy

mapping

Finisher static <T,A,R,RR> Collector<T,A,RR> collectingAndThen(Collector<T,A,R> downstream, Function<R,RR> finisher)

peek

Partitioning Results

tbd

```
    // Partition students into passing and failing
    Map<Boolean, List<Student>> passingFailing =
        students.stream()
                .collect(Collectors.partitioningBy(s -> s.getGrade() >= PASS_THRESHOLD)
```

Parallel Streams

Be aware these lose any staticly-held context, e.g., security context in a servlet.

Creation

```
Stream<String> parStream = list.parallelStream();
Stream<String> parStream = Stream.of(myArray).parallel();
```

unordered Can speed up the `limit` or `distinct`

```
stream.parallelStream().unordered().distinct();
```

PS: Work with the streams library. Eg. use `filter(x -> x.length() < 9)` instead of a `forEach` with an `if`.

http://docs.oracle.com/javase/8/docs/api/java/util/stream/package-summary.html#MutableReduction

Primitive-Type Streams

Wrappers (like Stream) are inefficients. It requires a lot of unboxing and boxing for each element. Better to use `IntStream`, `DoubleStream`, etc.

Creation

```
IntStream stream = IntStream.of(1, 2, 3, 5, 7);
stream = IntStream.of(myArray); // from an array
stream = IntStream.range(5, 80); // range from 5 to 80

Random gen = new Random();
IntStream rand = gen(1, 9); // stream of randoms
```

IntStream

```
IntStream.of(3, 1, 4);
//> 3, 1, 4

IntStream.rangeClosed(1, 5);
//> 1, 2, 3, 4, 5

IntStream.range(1, 5);
//> 1, 2, 3, 4

IntStream.iterate(1, i -> i * 2).limit(5);
```

```
//> 1, 2, 4, 8, 16

IntStream.generate(() -> ThreadLocalRandom.current().nextInt(10)).limit(5);
//> 5, 4, 9, 7, 1
```

Use *mapToX* (mapToObj, mapToDouble, etc.) if the function yields Object, double, etc. values. boxed() min() max()

Creating objects using Generic Types

Definition:

```
 T T[] toArray(IntFunction<T[]> constructor) {
        int n = ...; // length of array
        T[] result = constructor.apply(n);
        // ... populate result array here
        return result;
}
```

Calling

```
String[] array = MyClass.toArray(String[]::new);
```

Interface Default Methods

default methods. Useful for adding new methods to existing interfaces without breaking existing implementations.

```
interface Foo {
        default boolean isTrue() { return true; }
}
```

Collections

sort `sort(list, comparator)`

```
list.sort((a, b) -> a.length() - b.length())
list.sort(Comparator.comparing(n -> n.length())); // same
list.sort(Comparator.comparing(String::length)); // same
//> [Bohr, Tesla, Darwin, Newton, Galilei, Einstein]
```

removeIf

```
list.removeIf(w -> w.length() < 6);
//> [Darwin, Galilei, Einstein, Newton]
```

Merge `merge(key, value, remappingFunction)`

```
Map<String, String> names = new HashMap<>();
names.put("Albert", "Ein?");
names.put("Marie", "Curie");
names.put("Max", "Plank");

// Value "Albert" exists
// {Marie=Curie, Max=Plank, Albert=Einstein}
names.merge("Albert", "stein", (old, val) -> old.substring(0, 3) + val);

// Value "Newname" don't exists
```

```
// {Marie=Curie, Newname=stein, Max=Plank, Albert=Einstein}
names.merge("Newname", "stein", (old, val) -> old.substring(0, 3) + val);
```

Generic Static Methods

Definition

```
public class SomeStaticMethods {
    public static <T> boolean method1(T t) { ... }
    public static <T> boolean methodThatOnlyConsumes(List<? extends T> listT) { ... }
    public static <T> boolean methodThatOnlyAdds(List<? super T> listT) { ... }
    public static <T> boolean methodThatDoesBoth(List<T> listT) { ... }

    public static <T> T method3(boolean b) { ... }
    public static <T> List<? super T> method4(boolean b) { ... }

    public static <T,U> U method5(T t) { ... }
    public static <T,U> List<? super U> method6(List<? extends T> listT) { ... }
}
```

Invocation

```
SomeStaticMethods.<String>method();
SomeStaticMethods.<String,String>method();
``

## try-with-resources

Resource assigned in try that implements AutoCloseable has implicity finally "close()" c

```java
try (Stream<Path> paths = Files.list(directoryPath)) {

}
```

I/O Improvements – Files, Readers, etc

Working with files on disk

- Use a Path instead File. No need to specify file-system specific delimiters.
- Immutable, so great for concurrency.
- Construct with Paths

```
// non-cross platform construction
Path absoluteToDirectory2 = Paths.getPath("/usr/var/logs");

// cross platform construction
// "/" is the root, not the delimiter!
Path absoluteToDirectory = Paths.getPath("/", "usr", "var", "logs");
Path relativeToDirectory = Paths.getPath("logs");

Path absoluteToFile = Paths.getPath("/", "usr", "var", "logs", "access.log");
Path relativeToFile = Paths.getPath("logs", "access.log");

// windows?
Path absoluteToFile = Paths.getPath("c:\\", "usr", "var", "logs", "access.log");

```java
```

Resolve paths relative to other paths

```java
Path logPath = Paths.getPath("/usr/var/logs");
Path accessPartialPath = Paths.getPath("tomcat", "access.log");
Path accessPath = logPath.resolve(accessPartialPath);
// normalize tbd
// relativize tbd
```

Path#toFile File#toPath, oh yeah

Path path = Paths.getPath("logs", "access.log"); BufferedReader reader = Files.newBufferedReader(path, StandardCharsets.UTF_8);

```
byte[] bytes = Files.readAllBytes(path);
String fileString = new String(Files.readAllBytes(path), StandardCharsets.UTF_8);
List<String> lines = Files.readAllLines(path);
```

- static Path write(Path, Iterable<? extends CharSequence>, OpenOption...)

Files.write(path, content.getBytes(StandardCharsets.UTF_8)); // create or overwrite semantics
Files.write(path, content.getBytes(StandardCharsets.UTF_8), StandardOpenOption.APPEND);

- java.nio.file.Files
  - static BufferedReader newBufferedReader(Path)
  - static BufferedWriter newBufferedWriter(Path, OpenOption...)
  - InputStream in = Files.newInputStream(path);
  - OutputStream out = Files.newOutputStream(path);
  - copy(InputStream, Path)
  - copy(Path, OutputStream)

*Files Stream methods

- static Stream list(Path dir) -- list of Paths representing files and directories in a directory
- static Stream walk(Path start, FileVisitOption... options) -- walk a directory tree, with max depth
- static Stream walk(Path start, int maxDepth, FileVisitOption... options) -- walk a directory tree, with max depth
- static Stream find(Path start, int maxDepth, BiPredicate<Path, BasicFileAttributes> matcher, FileVisitOption... options)
- BufferedReader
  - lines() – same as Files.lines methods

FileVisitOption – only one FOLLOW_LINKS OpenOption

Files.lines

- static Stream lines(Path) -- lazy, defaults to UTF-8
- static Stream lines(Path path, Charset cs) stream of lines, one line per entry, but does not automatically close the File resource! So must wrap in a try()

```
Path path = FileSystems.getDefault().getPath("/usr", "var", "logs", "access.log"); // "/
try (Stream<String> lines = Files.lines(path)) {
   Optional<String> timeoutEntry = lines.filter(s -> s.startsWith("timeout:")).findAny()
   ...
}
```

Exception thrown as UncheckedIOException

Annotations

Collections and Collections

- Iterable
  - forEach
- Iterator
  - forEachRemaining
- Collection interface
  - boolean removeIf(Predicate<? super E> filter)
  - Stream stream()
  - Stream parallelStream()
  - Spliterator spliterator()
- Collections static class (tbd)
(unmodifiable	synchronized	checked	empty)Navigable(Set	Map)
  - checkedQueue
emptySorted(Set	Map)
empty(Set	Map)

- List class
  - void replaceAll(UnaryOperator operator)
  - void sort(Comparator<? super E> c)
- Map
  - forEach
  - replace
  - replaceAll
  - remove(key, value)
  - putIfAbsent
  - compute
  - computeIfAbsent
  - computeIfPresent
  - merge
- BitSet
  - Stream stream
- NavigableSet
- NavigableMap
- https://docs.oracle.com/javase/6/docs/api/java/util/concurrent/ConcurrentSkipListMap.html
- https://docs.oracle.com/javase/6/docs/api/java/util/concurrent/CopyOnWriteArraySet.html
- https://docs.oracle.com/javase/6/docs/api/java/util/SortedSet.html
- https://docs.oracle.com/javase/6/docs/api/java/util/concurrent/CopyOnWriteArrayList.html

Minor Changes

- Objects static class
  - static boolean isNull(Object)
  - static boolean nonNull(Object)
  - static T requireNonNull(T, Supplier) -- if not null, lazily generate an error message (complementary to eager requireNonNull(T) and requireNonNull(T, String) introduced in 7)
- String class
  - static String join(CharSequence delimiter, CharSequence... elements)
  - static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)
- Comparator

    - comparing(Function<? super T,? extends U> keyExtractor) – static method to create a Comparator typically converting T to Comparable like String or Integer
    - comparing(Function<? super T,? extends U> keyExtractor, Comparator<? super U> keyComparator) –
    - thenComparing(Function<? super T,? extends U> keyExtractor) – tie breaker
    - naturalOrder – Comparator that's the default for a Comparable type.
    - reverseOrder() - Returns a comparator that imposes the reverse of the natural ordering.
    - comparing(Function<? super T,? extends U> keyExtractor, Comparator<? super U> keyComparator)
comparing{Int	Double	Long}(To{Int	Double	Long}Function<? super T>)

    - nullsFirst(Comparator<? super T> comparator)
    - nullsLast(Comparator<? super T> comparator)
    - reversed() - Returns a comparator that imposes the reverse ordering of this comparator.
- Annotations? tbd

Numbers and Math

BigInteger

- longValueExact(), intValueExact(), shortValueExact(), byteValueExact() throws ArithmeticException

Primitive wrapper types

- hashCode() in native, so no boxing

New methods in primitive wrapper types:

Method	Boolean	Byte	Short	Integer	Long	Float	Double
sum()	—	—	—	yes	yes	yes	yes
max()	—	—	—	yes	yes	yes	yes
min()	—	—	—	yes	yes	yes	yes
logicalAnd(T)	yes	—	—	—	—	—	—
logicalOr(T)	yes	—	—	—	—	—	—
logicalXor(T)	yes	—	—	—	—	—	—
toUnsignedInt(T)	—	yes	yes	—	—	—	—
toUnsignedLong(T)	—	yes	yes	yes	—	—	—
compareUnsigned(T)	—	—	—	yes	yes	—	—
divideUnsigned(T)	—	—	—	yes	yes	—	—
remainderUnsigned(T)	—	—	—	yes	yes	—	—
#isFinite(T)	—	—	—	—	—	yes	yes

- Math.floorMod(x, n) – if x might be negative

Base64

- Base64
  - getEncoder / getDecoder - general Base64 encoder/decoder
  - getUrlEncoder / getUrlDecoder – URL and filename safe scheme
  - getMimeEncoder / getMimeDecoder – MIME type scheme
  - Encoder
    - String encodeToString(byte[]) {
    - byte[] encode(byte[])
    - int encode(byte[], byte[]) {
    - ByteBuffer encode(ByteBuffer)
    - OutputStream wrap(OutputStream)
    - Encoder withoutPadding()
  - Decoder
    - byte[] decode(byte[] src)
    - byte[] decode(String src)
    - int decode(byte[] src, byte[] dst)
    - ByteBuffer decode(ByteBuffer buffer)
    - InputStream wrap(InputStream is)

Locale https://docs.oracle.com/javase/8/docs/api/java/util/Locale.html

- Locale.forLanguageTag("en-US")
- Parts
  - Language - two or three lowercase letters, e.g., en, de, jp, zh
  - Script - four letters, capitalized, e.g., Latn (Latin), Cyrl (Cyrillic), Hant (traditional Chinese characters) – Serbian (Latin or Cyrillic), Chinese (traditional or simplified)
  - Country - the country-specific variant of a language. two uppercase letters or three digits, e.g., en_US (English, United States), de_CH (German, Switzerland), zh_CN (Chinese, Mainland China), zh_TW (Taiwan)
  - Variant – language and country specific variant, two uppercase letters, e.g., Nyorsk Norwegian. Rare, as many are now use a different Language code, e.g., Nyorsk Norwegian now uses nn_NO instead of no_NO_NY.
  - Extensions – local preference for calendar, number, etc. Examples: Japanese calendar (u-ca-japanese), Thai numerals (u-nu-thai). ja_JP_JP_#u-ca-japanese

LanguageRange List ranges = Stream.of("de_CH", "de", "*_CH") // prefer Swiss German, then any German, than any language with a Swiss country variation .map(Locale.LanguageRange::new) .collect(toList()); // A list containing the Locale.LanguageRange objects for the given strings List matches = Locale.filter(ranges, Arrays.asList(Locale.getAvailableLocales())); // The matching locales: de_CH, de, de_AT, de_LU, de_DE, de_LI, fr_CH, it_CH Locale bestMatch = Locale.lookup(ranges, locales);

JDBC

Java 7 - 4.1, Java 8 - 4.2

methods to convert java.sql.{Date	Time	Timestamp} <=> java.time.{LocalDate	LocalTime	LocalDateTime}

- **long Statement#executeLargeUpdate** - when row count exceeds Integer.MAX_VALUE.
- Statement and ResultSet
  - getObject(column, Class type)
  - setObject(column, Class type)

Date / Time

- Immutable objects, unlike Date and Calendar

- DateTimeFormatter – format and parse dates and times.
- Instant - Date equivalent – a single point on the time line
  - now() - current Instant
- Duration - delta between two instants
  - between(Instant, Instant) - difference between two Instants
to{Nanos	Millis	Seconds	Minutes	Hours	Days} - convert a duration to a long representing that unit

- Instant and Duration
  - plus(Duration), minus(Duration)
plus{Nanos	Millis	Seconds	Minutes	Hours	Days}(long), minus{Nanos	Millis	Seconds	Minutes	H

  - multipliedBy(long)
  - dividedBy(long)
  - negated
  - isZero
  - isNegative
- Period - used when advancing a timezoned time
- LocalDateTime - non-time zoned date/time
- ZonedDateTime - time zoned date/time (GregorianCalendar equivalent)
- TemporalAdjuster - calendar calculations, e.g., first Wednesday in June 2312

Etc

- Scanner
- DirectoryStream - J7, huge directory trees
- @Repeatable annotations
- annotation type use vs. only name use
- annotation Method Parameter Reflection
- Logger – all methods have a Supplier argument that lazily computes the error message
- Regex
  - Java 7 introduced named capturing groups.
  - Pattern.splitAsStream
  - Stream words = Pattern.compile("[\\P{L}]+").splitAsStream(contents);
  - Pattern#asPredicate – convert regex to predicate
- String contents = new String(Files.readAllBytes(path), StandardCharsets.UTF_8);
- this.foo = Objects.requireNonNull(foo); // throws NPE
- BitSet - set of integers that is implemented as a sequence of bits. The ith bit is set if the set contains the integer i. That makes for very efficient set operations. Union/intersection/complement are simple bitwise or/and/not.
- ProcessBuilder ProcessBuilder builder = new ProcessBuilder(“grep”, “-o”, “[A-Za-z_][A-Za-z_0-9]*”); builder.redirectInput(Paths.get(“Hello.java”).toFile()); builder.redirectOutput(Paths.get(“identifiers.txt”).toFile()); // or, builder.inheritIO() Process process = builder.start(); process.waitFor(1, TimeUnit.MINUTES);

Objects.equals(a, b) instead of a.equals(b) hashCode using Objects.hash(first, second, ..., last);

```
List<Long> ids = stream(result.results()).map(JO::getID).collect
groupFilter.setIDs(ids);
Map<Long,Group> map = stream(groupManager.getGroups(startIndex,
        .collect(toMap(Group::getID, identity(), (v1, v2) -> v1)
groups = ids.stream().map(map::get).collect(toList());
```

Phil Varner

Phil Varner
philvarner@gmail.com

philvarner
philvarner

Homepage of Phil Varner